

Singleton em Go explicado para leigos

By Antônio Marcelo

Singleton é um padrão de programação usado quando queremos garantir que exista apenas uma única instância de alguma coisa dentro de um programa. Em outras palavras, em vez de criar vários objetos iguais espalhados pelo sistema, criamos apenas um e todo mundo usa esse mesmo objeto.

Imagine uma escola com uma única secretária. Todos os alunos, professores e funcionários precisam passar pela mesma secretária para resolver documentos, horários e informações. Não faria sentido cada pessoa criar uma secretária diferente. No mundo da programação, o Singleton funciona de forma parecida: ele centraliza o acesso a um recurso único.

Em Go, isso pode ser útil quando temos algo que deve ser compartilhado por todo o programa, como uma configuração global, uma conexão com banco de dados, um logger ou um gerenciador de cache. A ideia é evitar desperdício de memória, repetição de código e comportamentos inconsistentes.

Um exemplo simples seria uma configuração do sistema:

```
package main

import "fmt"

type Config struct {
    AppName string
    Version string
}

var config *Config

func GetConfig() *Config {
    if config == nil {
        config = &Config{
            AppName: "Meu Sistema",
            Version: "1.0",
        }
    }

    return config
}

func main() {
    c1 := GetConfig()
    c2 := GetConfig()

    fmt.Println(c1.AppName)
    fmt.Println(c2.Version)

    fmt.Println(c1 == c2)
}
```

Nesse código, temos uma struct chamada `Config`, que guarda informações do sistema. A variável `config` fica fora da função principal e armazena a única instância da configuração.

A função `GetConfig()` verifica se a configuração já existe. Se ainda não existir, ela cria. Se já existir, ela apenas devolve a mesma configuração criada anteriormente.

No final, quando fazemos `c1 == c2`, o resultado será `true`, porque as duas variáveis apontam para o mesmo lugar na memória. Ou seja, não foram criadas duas configurações diferentes.

Porem, em Go, existe um detalhe importante: se o programa usa concorrência, ou seja, várias goroutines acessando a mesma função ao mesmo tempo, o exemplo acima pode dar problema. Duas goroutines poderiam tentar criar a configuração ao mesmo tempo.

Para resolver isso, Go oferece o `sync.Once`, que garante que determinado trecho de código seja executado apenas uma vez, mesmo com concorrência.

Veja uma versão mais segura:

```
package main

import (
    "fmt"
    "sync"
)

type Config struct {
    AppName string
    Version string
}

var config *Config
var once sync.Once

func GetConfig() *Config {
    once.Do(func() {
        config = &Config{
            AppName: "Meu Sistema",
            Version: "1.0",
        }
    })

    return config
}

func main() {
    c1 := GetConfig()
    c2 := GetConfig()

    fmt.Println(c1.AppName)
    fmt.Println(c2.Version)

    fmt.Println(c1 == c2)
}
```

Agora, mesmo que várias partes do programa chamem `GetConfig()` ao mesmo tempo, o `once.Do()` garante que a configuração será criada apenas uma vez.

O Singleton pode ser muito útil, mas deve ser usado com cuidado. Se tudo no sistema virar Singleton, o código pode ficar difícil de testar, difícil de modificar e muito dependente de estados globais. Por isso, ele deve ser usado apenas quando realmente faz sentido existir uma única instância compartilhada.

Um bom caso de uso em Go seria um logger global:

```
package main

import (
    "fmt"
    "sync"
)

type Logger struct{}
```

```

func (l *Logger) Log(msg string) {
    fmt.Println("LOG:", msg)
}

var logger *Logger
var once sync.Once

func GetLogger() *Logger {
    once.Do(func() {
        logger = &Logger{}
    })

    return logger
}

func main() {
    log := GetLogger()
    log.Log("Sistema iniciado")

    outroLog := GetLogger()
    outroLog.Log("Executando processo")

    fmt.Println(log == outroLog)
}

```

Nesse exemplo, o programa inteiro usa o mesmo logger. Isso evita criar varios loggers diferentes e centraliza a responsabilidade de registrar mensagens.

Resumindo: Singleton e um padrao usado para garantir que apenas uma instancia de algo exista no sistema. Em Go, a forma mais segura de implementar isso e usando `sync.Once`, principalmente quando existe concorrencia. Ele e util para configuracoes, logs, conexoes e recursos compartilhados, mas deve ser usado com moderacao.